

CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

GIẢI THUẬT TÌM KIẾM TUYẾN TÍNH & NHỊ PHÂN



TS. TRẦN NGỌC VIỆT
TH.S LÊ CÔNG HIẾU



HỌC KỲ 3 – NĂM HỌC 2023-2024



KHÓA K29 & K28



NỘI DUNG

- *Giới thiệu bài toán tìm kiếm**
- *Tìm kiếm tuyến tính**
- *Tìm kiếm nhị phân**



◇ Giải thuật (*algorithms*): đó là dãy các câu lệnh (*statements*) chặt chẽ và rõ ràng xác định trình tự các thao tác trên một số đối tượng nào đó sao cho sau một số hữu hạn bước thực hiện ta đạt được kết quả mong muốn.

◇ Thuật toán: là một dãy hữu hạn các bước, mỗi bước mô tả chính xác các phép toán hoặc hành động cần thực hiện, để giải quyết một vấn đề.

Niklous Wirth, cha đẻ của ngôn ngữ lập trình Pascal và kỹ thuật lập trình cấu trúc đã đúc kết ý nghĩa của dữ liệu và mối quan hệ hữu cơ của nó với giải thuật bằng mệnh đề nổi tiếng:

Chương trình = Thuật toán + Cấu trúc dữ liệu



1. Giới thiệu bài toán tìm kiếm

- ❖ Tìm kiếm là quá trình xác định một đối tượng nào đó trong một tập các đối tượng. Kết quả trả về:
 - Đối tượng tìm được (nếu có)
 - Xác định vị trí của đối tượng trong tập đó.
- ❖ Việc tìm kiếm dựa theo một trường nào đó của đối tượng, trường này là **khóa** (key) của việc tìm kiếm.

Ví dụ: Tìm sinh viên có họ tên x trong DSSV.

SV {MaSV, HoTen, DiaChi,...}

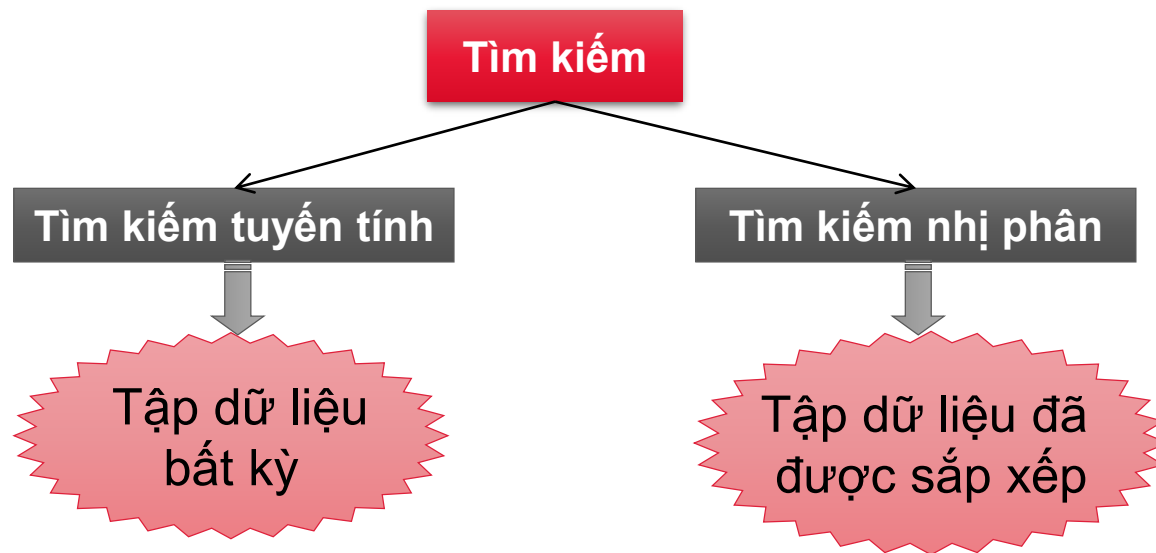
Khoá?

Kết quả trả về?

1. Giới thiệu bài toán tìm kiếm

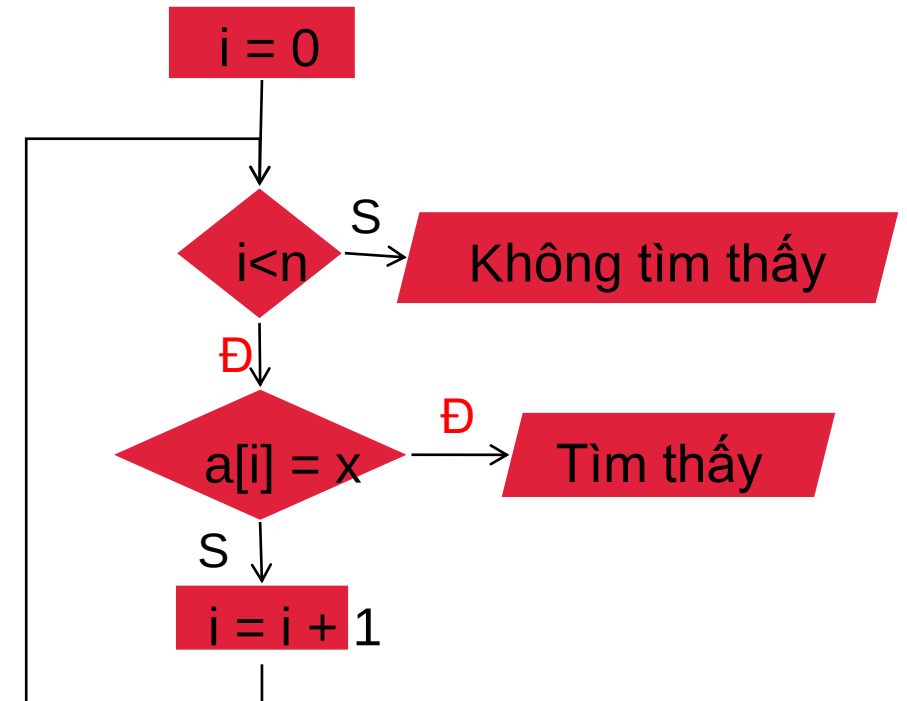
Bài toán được mô tả như sau:

- Tập dữ liệu được lưu trữ là dãy $a_1, a_2, \dots, a_n$. Giả sử chọn cấu trúc dữ liệu mảng để lưu trữ dãy số này trong bộ nhớ chính, có khai báo: `int a[n];`
- Khóa cần tìm là x : `int x;`



1. Giới thiệu bài toán tìm kiếm

- Ý tưởng: duyệt tuần tự từ phần tử đầu tiên, lần lượt so sánh **khóa tìm kiếm** với **khóa tương ứng** của các phần tử trong danh sách. Cho đến khi gặp phần tử cần tìm duyệt hết danh sách.
- Các bước tiến hành như sau:





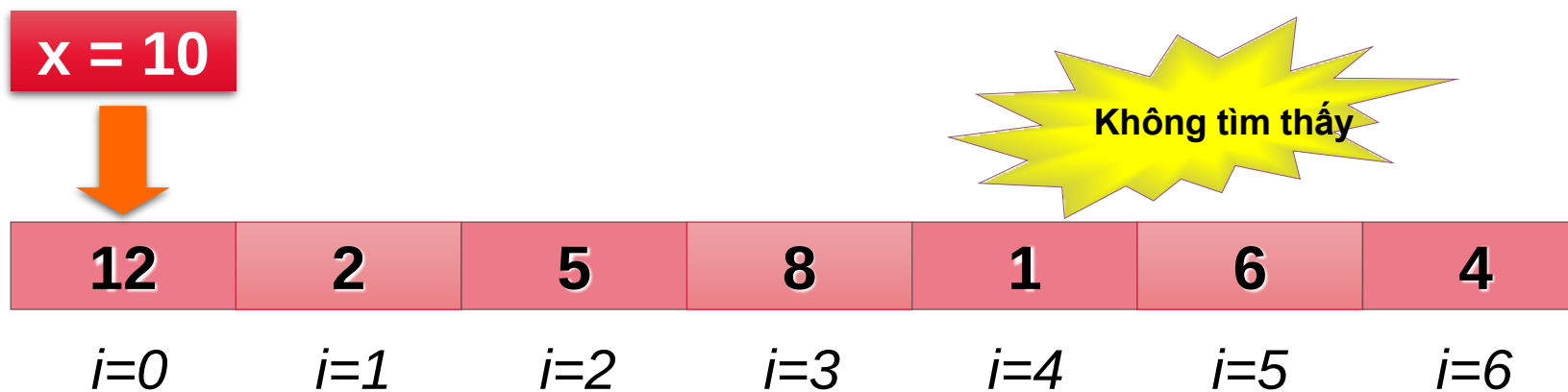
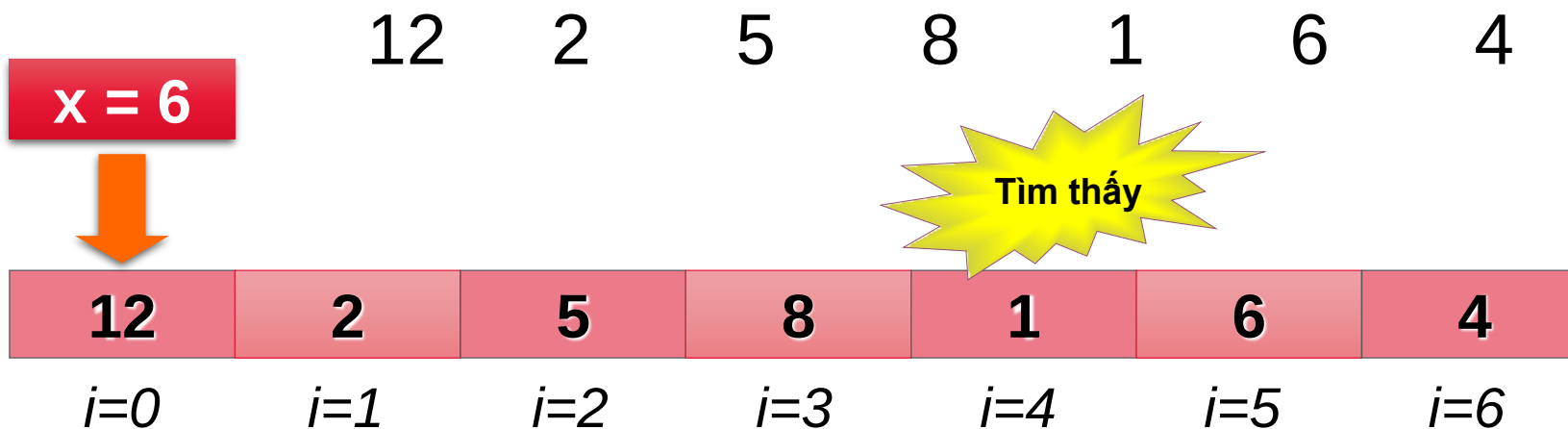
2. Tìm kiếm tuyến tính (Linear Search)

Ý tưởng:

Lần lượt so sánh x với phần tử thứ nhất, thứ hai, ... của mảng a cho đến khi gặp được phần tử cần tìm, hoặc hết mảng

2. Tìm kiếm tuyến tính

- **Ví dụ:** Cho dãy số a, giá trị tìm $x = 6$:





2. Tìm kiếm tuyến tính

Giải thuật

Bước 1:

$i = 0;$ // bắt đầu từ phần tử đầu tiên của dãy

Bước 2: So sánh $a[i]$ với x , có 2 khả năng :

- $a[i] = x$: Tìm thấy. Dừng kết thúc.
- $a[i] \neq x$: Chuyển sang Bước 3.

Bước 3:

- $i = i+1;$ // xét phần tử kế tiếp trong mảng; // hoặc ($i \geq n$)
- Nếu $i > n$: Hết mảng, không tìm thấy. Kết thúc.

Ngược lại, quay lui Bước 2.

*Áp dụng giải thuật tìm kiếm Tuyến tính

Ví dụ:

Cho mảng a , tìm khóa $x = 4$

12 - 2 - 5 - 8 - 1 - 6 - 4

B.1: $i=0$

B.2: $a[0]=12 \neq 4$

B.3: $i=0+1=1$

--

B.2: $a[1]=2 \neq 4$

B.3: $i=1+1=2$

.

Giải thuật

Bước 1:

$i = 0;$ // bắt đầu từ phần tử đầu tiên của dãy

Bước 2: So sánh $a[i]$ với x , có 2 khả năng :

- $a[i] = x$: Tìm thấy. Dừng kết thúc.
- $a[i] \neq x$: Chuyển sang Bước 3.

Bước 3:

- $i = i+1;$ // xét phần tử kế tiếp trong mảng
- Nếu $i > n$: Hết mảng, không tìm thấy. Dừng Ngược lại, quay lui Bước 2.

2. Tìm kiếm tuyến tính

Thuật toán tìm kiếm tuyến tính tựa ngôn ngữ C++ (tham khảo)

*//Trả về: vị trí xuất hiện đầu tiên của x trong mảng a
Trả về: -1 nếu x không có trong mảng a //*

```
int Search(int a[], int n, int key)
{
    int i =0;
    while (i<n && key != a[i]) //phep And
        i++;
    if (i < n)
        return i;                // tìm thấy tại vị trí i
    return -1;                    // tìm không thấy
}
```

2. Tìm kiếm tuyến tính

Thuật toán tìm kiếm tuyến tính cải tiến (tham khảo)

```
int Search(int a[], int n, int key)
{
    int i = 0;
    a[n] = key;           // thêm phần tử thứ n+1
    while (key != a[i])
        i++;
    if (i == n)
        return -1;
        // tìm hết mảng nhưng không có x

    return i;             // tìm thấy x tại vị trí i
}
```



2. Tìm kiếm tuyến tính

- Độ phức tạp tính toán : $T(n)=O(n)$
- Nhận xét:
 - ✓ Giải thuật tìm kiếm tuyến tính **không phụ thuộc vào thứ tự** của các phần tử trong mảng. Đây là phương pháp **tổng quát** để tìm kiếm trên một dãy bất kỳ
 - ✓ Một thuật toán có thể được cài đặt theo **nhiều cách khác nhau**, kỹ thuật cài đặt ảnh hưởng nhiều đến **tốc độ thực hiện**. Ví dụ như thuật toán Search cải tiến sẽ chạy nhanh hơn thuật toán trước do vòng lặp while chỉ so sánh một điều kiện...

2. Tìm kiếm tuyến tính

So sánh, đánh giá tính toán giữa 2 giải thuật

```
int Search(int a[], int n, int key)
{
    int i = 0;
    while (i < n && key != a[i])
        i++;
    if (i < n)
        return i;           // tìm thấy tại vị trí i
    return -1;              // tìm không thấy
}
```

```
int Search(int a[], int n, int key)
{
    int i = 0;
    a[n] = key;             // thêm phần tử thứ n+1
    while (key != a[i])
        i++;
    if (i == n)
        return -1;         // tìm hết mảng nhưng không có x

    return i;              // tìm thấy x tại vị trí i
}
```

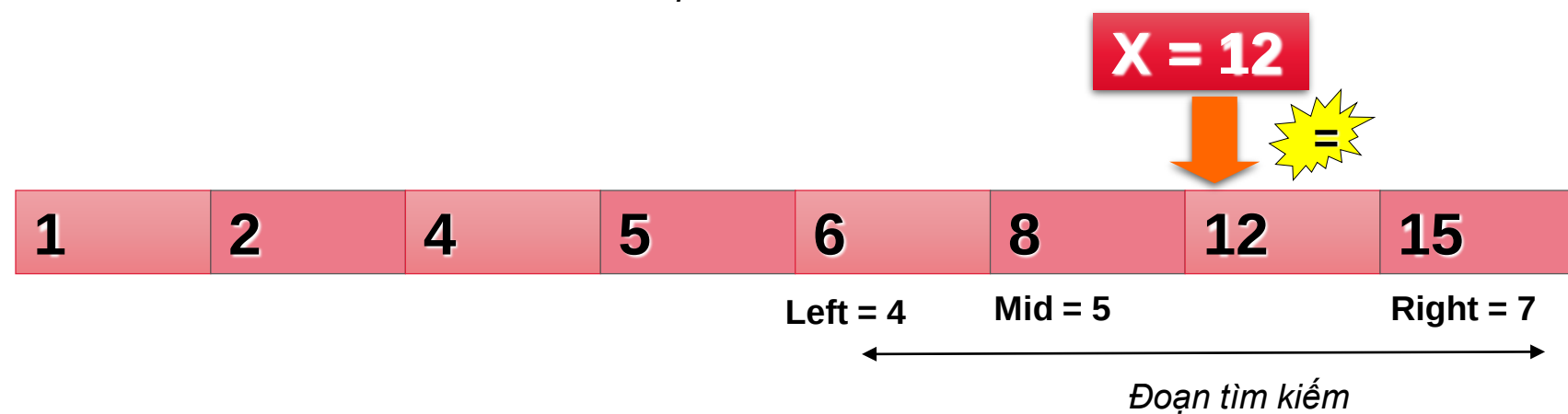


3. Tìm kiếm nhị phân (Binary Search)

Ý tưởng:

- Áp dụng đối với dãy số *có thứ tự*
- Mỗi bước tiến hành so sánh x với phần tử ở giữa của dãy hiện hành để quyết định phạm vi tìm kiếm kế tiếp

- 1 2 4 **X = 12** 6 8 12 15



3. Tìm kiếm nhị phân (Binary Search)

Giải thuật

Bước 1: $\text{left} = 0; \text{right} = n-1;$ //tìm kiếm tất cả các phần tử

Bước 2:

$\text{mid} = (\text{left} + \text{right}) / 2;$ //so sánh

So sánh $a[\text{mid}]$ với x , có 3 khả năng :

- $a[\text{mid}] = x$: Tìm thấy. Dừng kết thúc.
- $a[\text{mid}] > x$: //tìm tiếp x trong dãy con $a_{\text{left}} \dots a_{\text{mid}-1}$
 $\text{right} = \text{mid} - 1;$
- $a[\text{mid}] < x$: //tìm tiếp x trong dãy con $a_{\text{mid}+1} \dots a_{\text{right}}$
 $\text{left} = \text{mid} + 1;$

Bước 3:

Nếu $\text{left} \leq \text{right}$ //còn phần tử chưa xét tìm tiếp

Lặp lại Bước 2.

Ngược lại: Dừng kết thúc. //xét hết các phần tử trong mảng



3. Tìm kiếm nhị phân (Binary Search)

Ví dụ: Áp dụng giải thuật Tìm kiếm nhị phân. Mảng sắp xếp tăng dần.

Dãy số gồm 8 phần tử và $x = 12$ ($n = 8$)

left	mid					right	
1	2	4	5	6	8	12	15
i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7

B.1: $\text{left} = 0$; $\text{right} = n - 1 = 7$

B.2: $\text{mid} = (\text{left} + \text{right})/2 = (0 + 7)/2 = 3$ (lấy nguyên)

$\text{mid} = 3 \rightarrow a[\text{mid}] = a[3] = 5 < x = 12$:

$\text{left} = \text{mid} + 1 = 3 + 1 = 4$ ($\text{right} = 7$)

B.3: nếu $\text{left} < \text{right}$ ($4 < 7$)

				left	mid	right	
1	2	4	5	6	8	12	15
i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7

B.2: $\text{left} = 4$; $\text{right} = 7$

$\text{Mid} = (4 + 7)/2 = 5 \rightarrow a[\text{mid}] = a[5] = 8 < x = 12$:

$\text{left} = \text{mid} + 1 = 5 + 1 = 6$ ($\text{right} = 7$)

B.3: nếu $\text{left} < \text{right}$ ($6 < 7$)

					left = mid	right	
1	2	4	5	6	8	12	15
i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7

B.2: $\text{mid} = (\text{left} + \text{right})/2 = (6 + 7)/2 = 6$

$a[\text{mid}] = a[6] = 12 = x$. Kết thúc.

Giải thuật

Bước 1: $\text{left} = 0$; $\text{right} = n - 1$; //tìm kiếm tất cả các phần tử

Bước 2:

$\text{mid} = (\text{left} + \text{right})/2$; //so sánh

So sánh $a[\text{mid}]$ với x , có 3 khả năng :

$a[\text{mid}] = x$: Tìm thấy. Dừng kết thúc.

$a[\text{mid}] > x$: //tìm tiếp x trong dãy con $a_{\text{left}} \dots a_{\text{mid} - 1}$

$\text{right} = \text{mid} - 1$;

$a[\text{mid}] < x$: //tìm tiếp x trong dãy con $a_{\text{mid} + 1} \dots a_{\text{right}}$

$\text{left} = \text{mid} + 1$;

Bước 3:

Nếu $\text{left} \leq \text{right}$ //còn phần tử chưa xét tìm tiếp

Lặp lại Bước 2.

Ngược lại: Dừng kết thúc. //xét hết các phần tử trong mảng

3. Tìm kiếm nhị phân (Binary Search)

Ví dụ: Áp dụng giải thuật tìm kiếm nhị phân – Binary Search. Mảng sắp xếp tăng dần. Dãy số gồm 8 phần tử $x=12$ ($n=8$)

1	2	4	5	6	8	12	15
---	---	---	---	---	---	----	----

Giải:

Ta có

left			mid				right
1	2	4	5	6	8	12	15
i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7

B.1: $\text{left} = 0$; $\text{right} = n - 1 = 7$

B.2: $\text{mid} = (\text{left} + \text{right})/2 = (0 + 7)/2 = 3$ (lấy nguyên)

$\text{mid} = 3 \rightarrow a[\text{mid}] = a[3] = 5 < x = 12$:

$\text{left} = \text{mid} + 1 = 3 + 1 = 4$ ($\text{right} = 7$)

B.3: nếu $\text{left} < \text{right}$ ($4 < 7$)

				left	mid		right
1	2	4	5	6	8	12	15
i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7

B.2: $\text{left} = 4$; $\text{right} = 7$

$\text{Mid} = (4 + 7)/2 = 5 \rightarrow a[\text{mid}] = a[5] = 8 < x = 12$:

$\text{left} = \text{mid} + 1 = 5 + 1 = 6$ ($\text{right} = 7$)

B.3: nếu $\text{left} < \text{right}$ ($6 < 7$)

						left = mid	right
1	2	4	5	6	8	12	15
i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7

B.2: $\text{mid} = (\text{left} + \text{right})/2 = (6 + 7)/2 = 6$

$a[\text{mid}] = a[6] = 12 = x$. Kết thúc.

3. Tìm kiếm nhị phân (Binary Search)

Thuật toán tìm kiếm nhị phân tựa ngôn ngữ C (tham khảo)

```
int BinarySearch(int a[], int n, int key) {  
    int left = 0, right = n-1, mid;  
    while (left <= right) {  
        mid = (left + right) / 2;    // lấy điểm giữa  
        if (a[mid] == key)           // nếu tìm được  
            return mid;  
        if (a[mid] < key)             // tìm đoạn bên phải mid  
            left = mid+1;  
        else  
            right = mid-1;           // tìm đoạn bên trái mid  
    }  
    return -1;                       // không tìm được  
}
```

3. Tìm kiếm nhị phân (Binary Search)

Ví dụ 1: Áp dụng giải thuật Tìm kiếm nhị phân-Binary Search.

Cho mảng gồm 10 phần tử đã sắp xếp tăng dần.

Ta có

$n=10$; $x=23 = \text{key}$

$i=0$	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$	$i=6$	$i=7$	$i=8$	$i=9$
2	5	8	12	16	23	38	56	72	91

3. Tìm kiếm nhị phân (Binary Search)



2	5	8	12	16	23	38	56	72	91
L=0			mid=4			R=9			

+Khi $L \leq R$

$mid = (0+9)/2 = 4$ ($a[mid] = a[4] = 16$)

Nếu $a[mid] = 16 < key = 23$

$L = mid + 1 = 5$ ($a[5] = 23$)

3. Tìm kiếm nhị phân (Binary Search)

2	5	8	12	16	23	38	56	72	91
L = 5					mid = 7			R = 9	

+Khi $L \leq R$

$mid = (5+9)/2 = 7$ ($a[mid] = a[7] = 56$)

Nếu $a[mid] = 56 > key = 23$

$R = mid - 1 = 7 - 1 = 6$ ($a[6] = 38$)



3. Tìm kiếm nhị phân (Binary Search)

2	5	8	12	16	23	38	56	72	91
---	---	---	----	----	----	----	----	----	----

L = 5

R = 6

+ Khi $L \leq R$

$mid = (5+6)/2 = 5$ ($a[mid] = a[5] = 23$)

Nếu $a[mid] = a[5] = 23 = key$

Return mid. Kết thúc.

2	5	8	12	16	23	38	56	72	91
---	---	---	----	----	----	----	----	----	----

x=23

Ví dụ 2:
Cho mản

3. Tìm kiếm nhị phân (Binary Search)

+Khi $L \leq R$

$$\text{mid} = (0+3)/2 = 1 \quad (a[\text{mid}] = a[1] = 5)$$

Nếu $a[\text{mid}] = 5 < \text{key} = 8$

$$L = \text{mid} + 1 = 1 + 1 = 2$$

2	5	8	12	16	23	38	56	72	91
L=2			R=3						

+Khi $L \leq R$

$$\text{mid} = (2+3)/2 = 2 \quad (a[\text{mid}] = a[2] = 8)$$

Nếu $a[\text{mid}] = \text{key} = 8$

Return mid. Kết thúc.

2	5	8	12	16	23	38	56	72	91
x=8									



3. Tìm kiếm nhị phân (Binary Search)

- **Độ phức tạp tính toán cấp n:** $T(n) = O(\log_2 n)$
- **Nhận xét:**

Thuật toán nhị phân dựa vào quan hệ giá trị của các phần tử trong mảng để định hướng trong quá trình tìm kiếm, do vậy chỉ áp dụng được với dãy đã có thứ tự.

Thuật toán nhị phân tìm kiếm nhanh hơn tìm kiếm tuyến tính.

Tuy nhiên khi áp dụng thuật toán nhị phân thì cần phải quan tâm đến chi phí cho việc sắp xếp mảng. Vì khi mảng được sắp thứ tự rồi thì mới tìm kiếm nhị phân.

BÀI KIỂM TRA SỐ 2

Câu 1: Áp dụng giải thuật tìm kiếm nhị phân- Mảng các phần tử sắp xếp tăng dần

a)

$x=38 = \text{key}$, $n=10$

b)

$x=5 = \text{key}$, $n=10$

i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8	i=9
2	5	8	12	16	23	38	56	72	91



4. Bài tập lý thuyết

Câu 1. Cho dãy số sau:

3	4	6	6	12	16	21	34	41	80
1	2	3	4	5	6	7	8	9	10

Cho biết vị trí tìm thấy và số lần so sánh để tìm được phần tử có giá trị $x = 6$ khi áp dụng giải thuật tìm kiếm: tuyến tính và nhị phân.

Câu 2. Xây dựng giải thuật tìm kiếm phần tử có giá trị nhỏ nhất trong dãy số: Dùng mã tự nhiên, mã giả và lưu đồ.



4. Bài tập lý thuyết - Hướng dẫn

Câu 1: Tìm $x = 6$

3	4	6	6	12	16	21	34	41	80
1	2	3	4	5	6	7	8	9	10

Tìm x theo phương pháp tuyến tính

Vị trí	So sánh với x	Kết quả
$i=1$	$(a[i]=3) \neq x$	Tăng vị trí i
$i=2$	$(a[i]=4) \neq x$	Tăng vị trí i
$i=3$	$(a[i]=6) = x$	Dừng, trả về vị trí i (3)

4. Bài tập lý thuyết – Hướng dẫn

Câu 1: Tìm $x = 6$

3	4	6	6	12	16	21	34	41	80
1	2	3	4	5	6	7	8	9	10

Tìm x theo phương pháp nhị phân

Xét đoạn	Vị trí giữa đoạn	So sánh với x	Kết quả
$l=1, r=10$	$m=(l+r)/2=5$	$(a[m]=12) \neq x$	Cập nhật $r=m-1=4$ ($a[m] > x \rightarrow$ xét trái)
$l=1, r=4$	$m=(l+r)/2=2$	$(a[m]=4) \neq x$	Cập nhật $l=m+1=3$ ($a[m] < x \rightarrow$ xét phải)
$l=3, r=4$	$m=(l+r)/2=3$	$(a[m]=6) = x$	Dừng, trả về vị trí m (3)



4. Bài tập lý thuyết – Hướng dẫn

- **Input:** Mảng số nguyên a , kích thước n
- **Output:** vị trí phần tử có giá trị nhỏ nhất
- **Mã tự nhiên:**

Bước 1:

$i=2, vt=1;$

Bước 2:

Nếu $i > N$ thì trả về giá trị vt , kết thúc;

Bước 3:

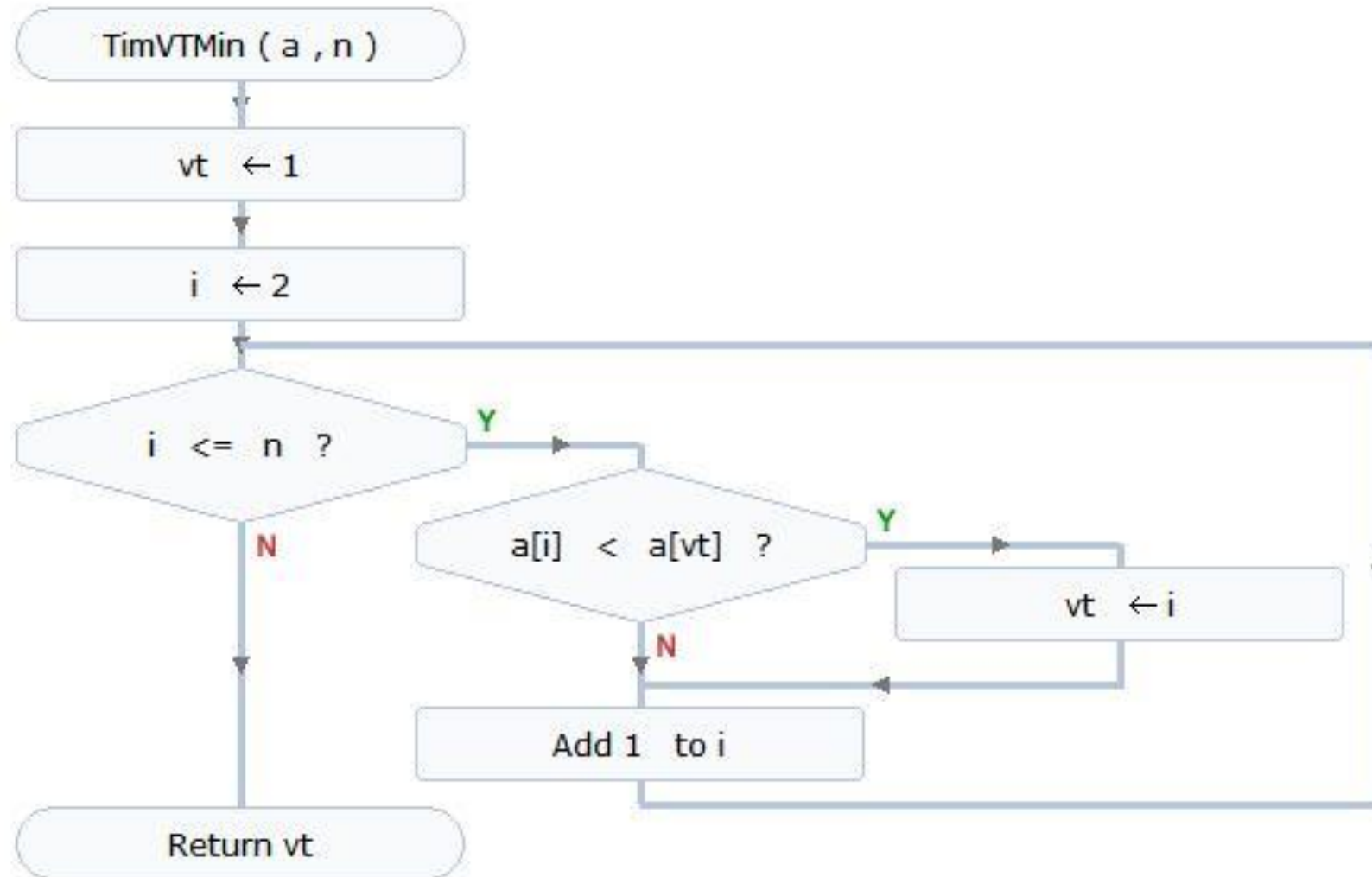
Nếu $a[i] < a[vt]$ thì $vt=i;$

$i=i+1;$

Lặp lại Bước 2;

4. Bài tập lý thuyết – Hướng dẫn

Flow Chart:





BÀI TẬP THỰC HÀNH

1. Sinh mảng ngẫu nhiên gồm N số nguyên có giá trị $\in (-100, 100)$
 - Tìm phần tử có giá trị X trong mảng bằng 2 phương pháp: Tìm tuyến tính và tìm nhị phân
2. Cho cấu trúc Sách (Mã sách: char[10], Tên sách: char[40], Giá: long).
Viết chương trình thực hiện:
 - Nhập, xuất danh sách gồm N cuốn sách.
 - Tìm cuốn sách có mã là X bằng phương pháp tìm kiếm tuần tự (X nhập từ bàn phím).
 - Tìm cuốn sách có mã là X bằng phương pháp tìm kiếm nhị phân. (X nhập từ bàn phím).
 - Liệt kê thông tin các cuốn sách có giá $> G$ (G nhập từ bàn phím).
 - Tìm cuốn sách có giá lớn nhất.

THỰC HÀNH

#Bài Thực hành 01.1:

```
>>> def tuyen_tinh(array, n, x):  
  
    for i in range(0, n):  
        if (array[i] == x):  
            return i  
    return -1  
  
>>> array = [20, 30, 15, 5, 10, 40]  
>>> x = 40  
>>> n = len(array)  
>>> result = tuyen_tinh(array, n, x)  
>>> print("Phan tu tim thay duoc tai vi tri la:", result)
```

#Nhan xet: input & output của Giai thuat tren??

#Nhắc lại kiến thức – Giải thuật tuyen tinh

Giải thuật

Bước 1:

$i = 0$; // bắt đầu từ phần tử đầu tiên của dãy

Bước 2: So sánh $a[i]$ với x , có 2 khả năng :

- $a[i] = x$: Tìm thấy. Dừng kết thúc.
- $a[i] \neq x$: Chuyển sang Bước 3.

Bước 3:

- $i = i + 1$; // xét phần tử kế tiếp trong mảng
- Nếu $i > n$: Hết mảng, không tìm thấy. Dừng Ngược lại, quay lui Bước 2.

THỰC HÀNH

#Bài Thực hành 01.2:

Input : array = [15, 25, 80, 30, 60, 50,
110, 100, 130, 180]

x = 110;

Output : 6

Phan tu tim thay duoc tai vi tri la: 6

Input : array = [15, 25, 80, 30, 60, 50,
110, 100, 130, 180]

x = 185;

Output : -1

Phan tu ko duoc tim thay trong arr[]



THỰC HÀNH

#Bài Thực hành 01.3:

```
def linearsearch(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1  
  
arr = ['Bao', 'An', 'Dat', 'Duc', 'Hung', 'Phi', 'Vinh', 'Dung']  
key = 'Phi'  
  
print("vị trí tìm thấy thứ i là: " +str(linearsearch(arr,key)))  
  
#Nhan xet: input & output cua Giai thuat tren??  
#Nhắc lại kiến thức - Giải thuật tuyến tính
```

THỰC HÀNH

#Bài Thực hành 02.1:

```
>>> import math
>>> def binary_search(array, element):
    mid = 0
    start = 0
    end = len(array)
    step = 0

    while (start <= end):
        step = step+1
        mid = (start + end) // 2

        if element == array[mid]:
            return mid

        if element < array[mid]:
            end = mid - 1
        else:
            start = mid + 1
    return -1

>>> array = [0, 4, 5, 9, 13, 15, 18, 24, 28, 29, 35]
>>> element = 28
>>> result = binary_search(array, element)
>>> print("Phan tu tim kiem duoc la:", result)

#Nhan xet: input & output cua Giai thuat tren ??
#Nhắc lại kiến thức - Giải thuật nhị phân
```

Giải thuật

Bước 1: left = 0; right = n-1; //tìm kiếm tất cả các phần tử

Bước 2:

mid = (left+right)/2; //so sánh

So sánh a[mid] với x, có 3 khả năng :

- a[mid] = x: Tìm thấy. Dừng kết thúc.
- a[mid] > x: //tìm tiếp x trong dãy con $a_{left} \dots a_{mid-1}$
right = mid - 1;
- a[mid] < x: //tìm tiếp x trong dãy con $a_{mid+1} \dots a_{right}$
left = mid + 1;

Bước 3:

Nếu left <= right //còn phần tử chưa xét tìm tiếp

Lặp lại Bước 2.

Ngược lại: Dừng kết thúc. //xét hết các phần tử trong mảng

THỰC HÀNH

#Bài Thực hành 02.2:

```
>>> import math
>>> def binary_search(arr, key):    #??
    mid = 0                        #??
    left = 0
    right = len(arr)
    step = 0
    while (left <= right):          #??
        step = step+1
        mid = (left + right) // 2

        if (key == arr[mid]):      #??
            return mid

        if (key < arr[mid]):        #??
            right = mid - 1
        else:
            left = mid + 1
    return -1

>>> arr = [0, 4, 5, 9, 13, 15, 18, 24, 28, 29, 35]
>>> key = 40
>>> result = binary_search(arr, key)
>>> print("Vi tri tim kiem thu i la:", result)

#Nhan xet: input & output cua Giai thuat Nhị phân ??
#Yêu cầu – comment vào dấu note, bước lặp, vòng lặp của giải thuật
```

THỰC HÀNH

#Bài Thực hành 02.3:

#Binary Search

```
def binary_search(arr, left, right, key):  
  
    if (right >= left):  
  
        mid = (left + right) // 2  
  
        if (arr[mid] == key):  
            return mid  
  
        elif (arr[mid] > key):  
            return binary_search(arr, left, mid - 1, key)  
  
        else:  
            return binary_search(arr, mid + 1, right, key)  
  
    else:  
        # không tìm thấy  
        return -1  
  
arr = [ 1, 3, 9, 10, 40, 52, 55, 180, 250, 350 ]  
key = 250  
result = binary_search(arr, 0, len(arr)-1, key)  
  
if (result != -1):  
    print("vi tri tim thay thu i la:", str(result))  
else:  
    print("khong tim thay phan tu trong mang")
```




TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

